

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

df = pd.read_csv('/content/Automobile_data.csv')

df.head()

{"type": "dataframe", "variable_name": "df"}

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 205 entries, 0 to 204
Data columns (total 26 columns):
#   Column                Non-Null Count  Dtype
---  -
0   symboling              205 non-null   int64
1   normalized-losses      205 non-null   object
2   make                   205 non-null   object
3   fuel-type              205 non-null   object
4   aspiration             205 non-null   object
5   num-of-doors           205 non-null   object
6   body-style             205 non-null   object
7   drive-wheels           205 non-null   object
8   engine-location        205 non-null   object
9   wheel-base             205 non-null   float64
10  length                 205 non-null   float64
11  width                  205 non-null   float64
12  height                 205 non-null   float64
13  curb-weight            205 non-null   int64
14  engine-type            205 non-null   object
15  num-of-cylinders       205 non-null   object
16  engine-size            205 non-null   int64
17  fuel-system            205 non-null   object
18  bore                   205 non-null   object
19  stroke                 205 non-null   object
20  compression-ratio      205 non-null   float64
21  horsepower             205 non-null   object
22  peak-rpm               205 non-null   object
23  city-mpg               205 non-null   int64
24  highway-mpg            205 non-null   int64
25  price                  205 non-null   object
dtypes: float64(5), int64(5), object(16)
memory usage: 41.8+ KB

df.describe()

```

```

{"summary":{"\n  \"name\": \"df\",\n  \"rows\": 8,\n  \"fields\": [\n    {\n      \"column\": \"symboling\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 72.18619738012798,\n        \"min\": -2.0,\n        \"max\": 205.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          0.8341463414634146,\n          1.0,\n          205.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      {\n        \"column\": \"wheel-base\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 54.121837757672914,\n          \"min\": 6.021775685025572,\n          \"max\": 205.0,\n          \"num_unique_values\": 8,\n          \"samples\": [\n            98.75658536585367,\n            97.0,\n            205.0\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        },\n        {\n          \"column\": \"length\",\n          \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": 62.57530527897738,\n            \"min\": 12.337288526555186,\n            \"max\": 208.1,\n            \"num_unique_values\": 8,\n            \"samples\": [\n              174.04926829268288,\n              173.2,\n              205.0\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n          },\n          {\n            \"column\": \"width\",\n            \"properties\": {\n              \"dtype\": \"number\",\n              \"std\": 57.05631291349402,\n              \"min\": 2.1452038526871817,\n              \"max\": 205.0,\n              \"num_unique_values\": 8,\n              \"samples\": [\n                65.90780487804878,\n                65.5,\n                205.0\n              ],\n              \"semantic_type\": \"\",\n              \"description\": \"\"\n            },\n            {\n              \"column\": \"height\",\n              \"properties\": {\n                \"dtype\": \"number\",\n                \"std\": 58.952696962027126,\n                \"min\": 2.4435219699049044,\n                \"max\": 205.0,\n                \"num_unique_values\": 8,\n                \"samples\": [\n                  53.72487804878049,\n                  54.1,\n                  205.0\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\"\n              },\n              {\n                \"column\": \"curb-weight\",\n                \"properties\": {\n                  \"dtype\": \"number\",\n                  \"std\": 1270.6950351955225,\n                  \"min\": 205.0,\n                  \"max\": 4066.0,\n                  \"num_unique_values\": 8,\n                  \"samples\": [\n                    2555.5658536585365,\n                    2414.0,\n                    205.0\n                  ],\n                  \"semantic_type\": \"\",\n                  \"description\": \"\"\n                },\n                {\n                  \"column\": \"engine-size\",\n                  \"properties\": {\n                    \"dtype\": \"number\",\n                    \"std\": 90.36221973782494,\n                    \"min\": 41.642693438179826,\n                    \"max\": 326.0,\n                    \"num_unique_values\": 8,\n                    \"samples\": [\n                      126.90731707317073,\n                      120.0,\n                      205.0\n                    ],\n                    \"semantic_type\": \"\",\n                    \"description\": \"\"\n                  },\n                  {\n                    \"column\": \"compression-ratio\",\n                    \"properties\": {\n                      \"dtype\": \"number\",\n                      \"std\": 69.11177355262139,\n                      \"min\": 3.972040321863297,\n                      \"max\": 205.0,\n                      \"num_unique_values\": 8,\n                      \"samples\": [\n                        10.142536585365855,\n                        9.0,\n                        205.0\n                      ],\n                      \"semantic_type\": \"\",

```

```

"description\": \"\"\\n      }\\n    },\\n    {\\n      \"column\\\":
\"city-mpg\\\",\\n      \"properties\\\": {\\n        \"dtype\\\":
\"number\\\",\\n        \"std\\\": 65.28506994468985,\\n        \"min\\\":
6.54214165300162,\\n        \"max\\\": 205.0,\\n
\"num_unique_values\\\": 8,\\n        \"samples\\\": [\\n
25.21951219512195,\\n        24.0,\\n        205.0\\n        ],\\n
\"semantic_type\\\": \"\",\\n        \"description\\\": \"\"\\\"\\n      }\\
n    },\\n    {\\n      \"column\\\": \"highway-mpg\\\",\\n
\"properties\\\": {\\n        \"dtype\\\": \"number\\\",\\n        \"std\\\":
64.0390049599267,\\n        \"min\\\": 6.88644313094182,\\n
\"max\\\": 205.0,\\n        \"num_unique_values\\\": 8,\\n
\"samples\\\": [\\n        30.75121951219512,\\n        30.0,\\n
205.0\\n        ],\\n        \"semantic_type\\\": \"\",\\n
\"description\\\": \"\"\\\"\\n      }\\n    }\\n  ]\\n}\", \"type\": \"dataframe\"}

```

```

df['normalized-losses'].replace('?', np.nan, inplace=True)
df['bore'].replace('?', np.nan, inplace=True)
df['stroke'].replace('?', np.nan, inplace=True)
df['horsepower'].replace('?', np.nan, inplace=True)
df['peak-rpm'].replace('?', np.nan, inplace=True)
df['price'].replace('?', np.nan, inplace=True)

```

```
df.isnull().sum()
```

symboling	0
normalized-losses	41
make	0
fuel-type	0
aspiration	0
num-of-doors	0
body-style	0
drive-wheels	0
engine-location	0
wheel-base	0
length	0
width	0
height	0
curb-weight	0
engine-type	0
num-of-cylinders	0
engine-size	0
fuel-system	0
bore	4
stroke	4
compression-ratio	0
horsepower	2
peak-rpm	2
city-mpg	0
highway-mpg	0

```

price                4
dtype: int64

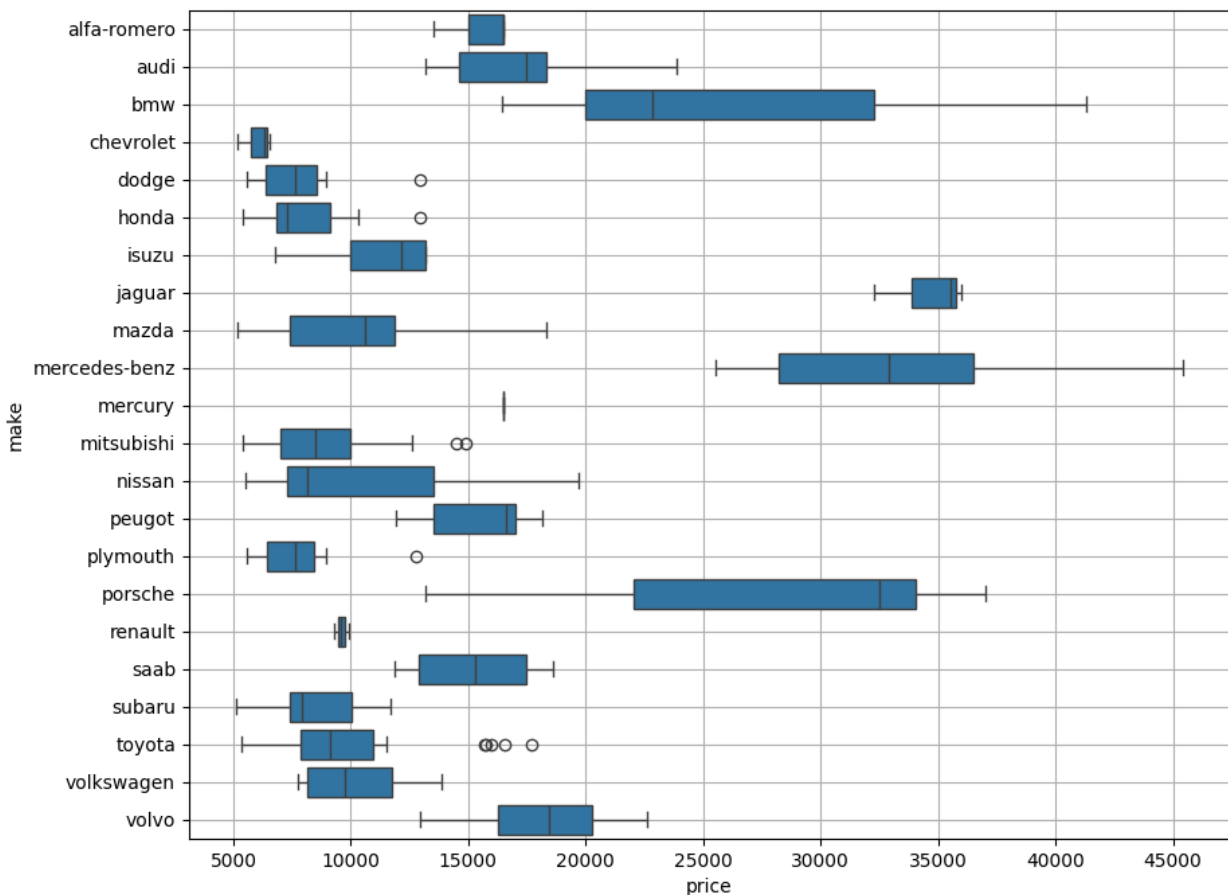
from sklearn.impute import SimpleImputer
si = SimpleImputer(missing_values=np.nan, strategy='mean')
df[['normalized-losses', 'bore', 'stroke', 'horsepower', 'peak-rpm', 'price']] = si.fit_transform(df[['normalized-losses', 'bore', 'stroke', 'horsepower', 'peak-rpm', 'price']])

df['num-of-doors'] = df['num-of-doors'].replace('?', 'four')

features = df.iloc[:, :-1]
target = df.iloc[:, -1]

plt.figure(figsize=(10, 8))
sns.boxplot(data=features, x=target, y='make')
plt.grid()

```



```

i = features[(features.make == 'dodge') & (target > 10000)].index
features.drop(i, axis=0, inplace=True)
target.drop(i, axis=0, inplace=True)

```

```

i = features[(features.make == 'honda') & (target > 12000)].index
features.drop(i, axis = 0, inplace = True)
target.drop(i, axis = 0, inplace = True)

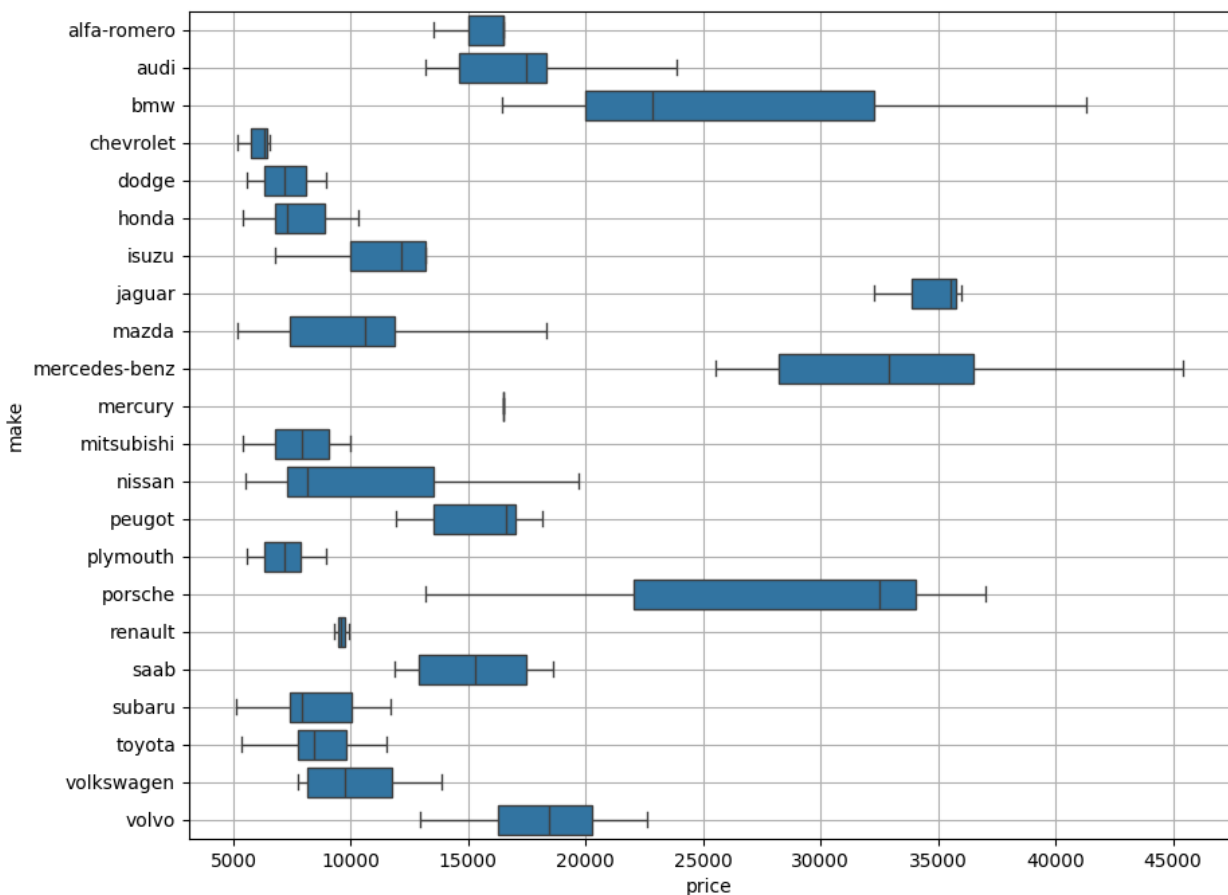
i = features[(features.make == 'mitsubishi') & (target > 12500)].index
features.drop(i, axis = 0, inplace = True)
target.drop(i, axis = 0, inplace = True)

i = features[(features.make == 'plymouth') & (target > 10000)].index
features.drop(i, axis = 0, inplace = True)
target.drop(i, axis = 0, inplace = True)

i = features[(features.make == 'toyota') & (target > 15000)].index
features.drop(i, axis = 0, inplace = True)
target.drop(i, axis = 0, inplace = True)

plt.figure(figsize=(10,8))
sns.boxplot(data=features, x=target, y='make')
plt.grid()

```



```

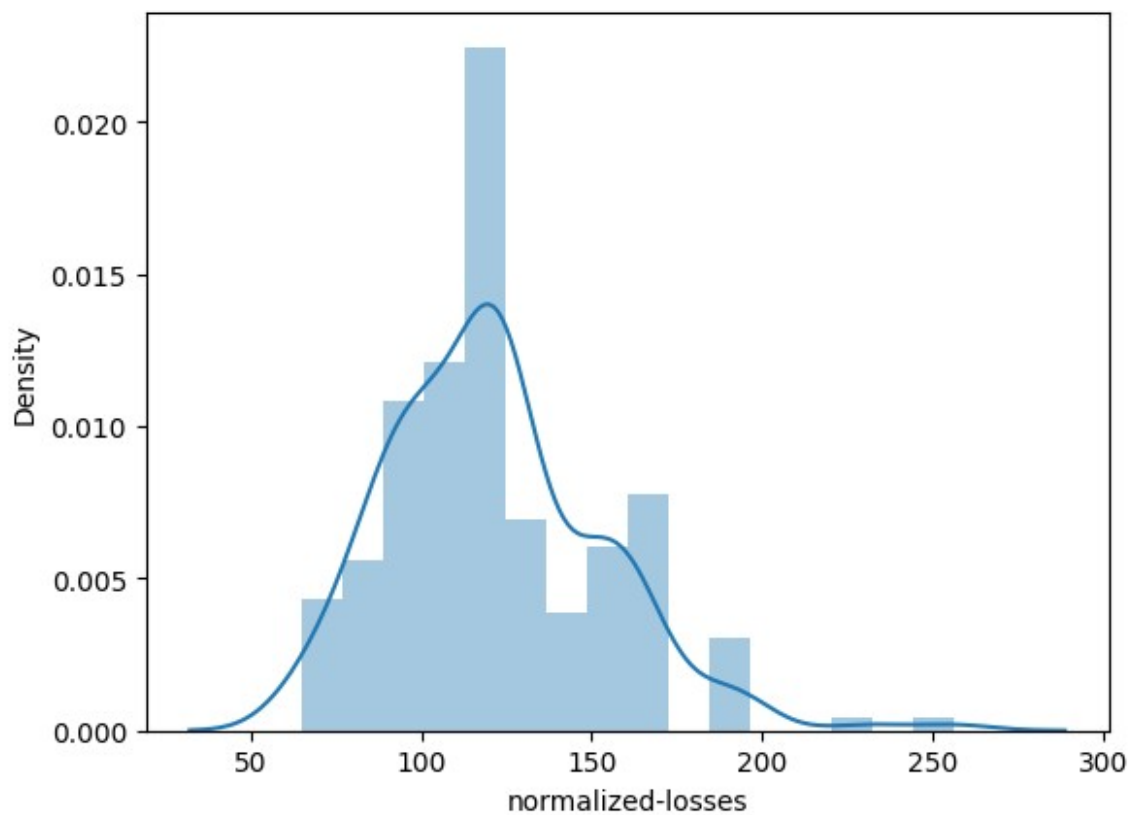
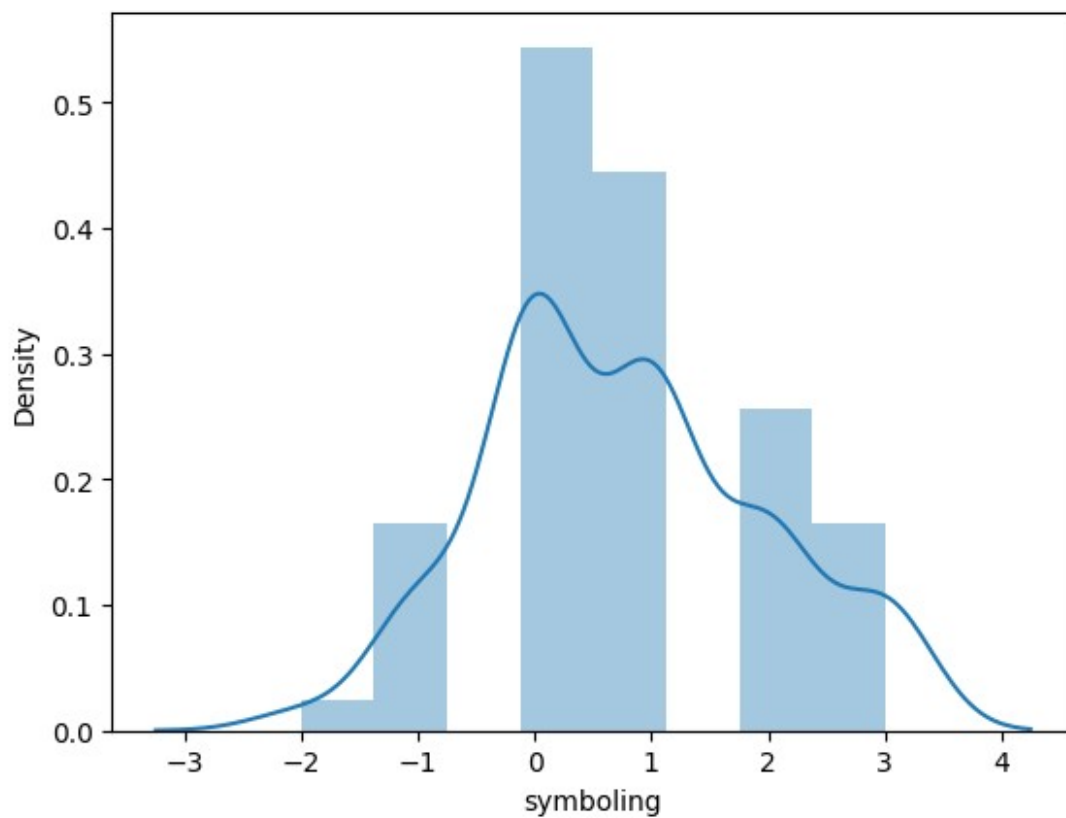
colnames = features.select_dtypes(['int64', 'float64']).columns
colnames

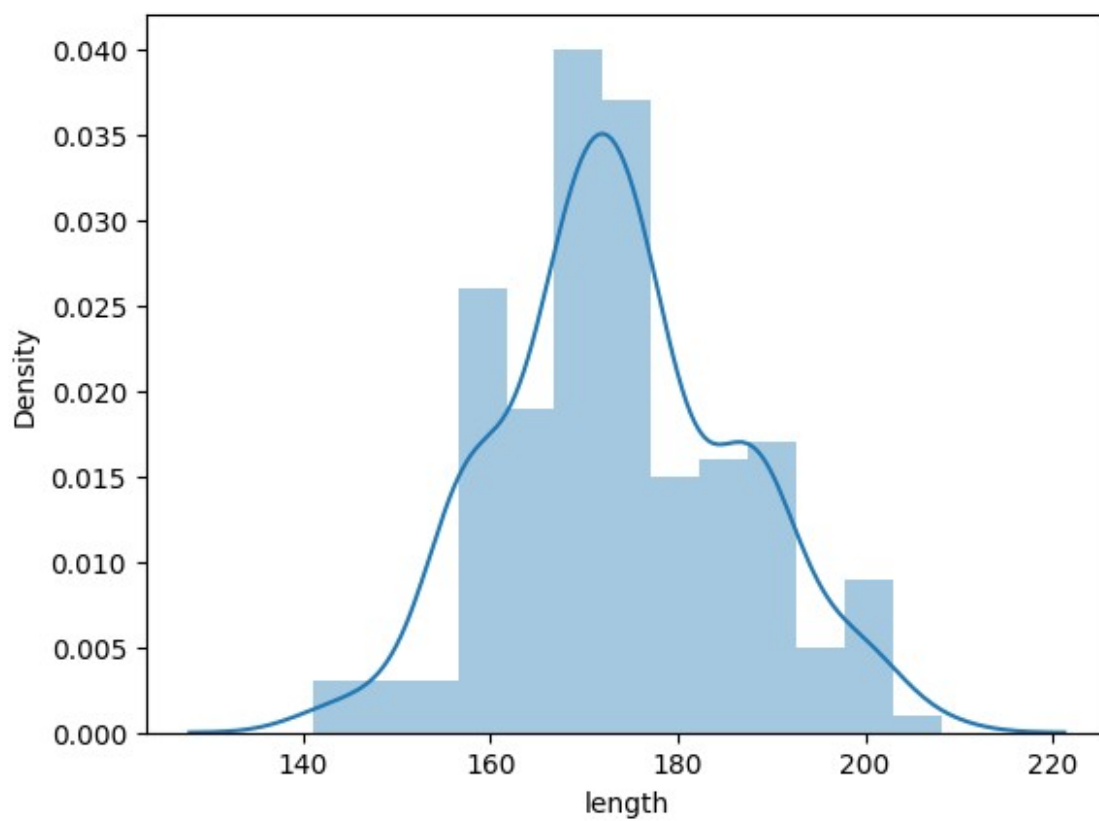
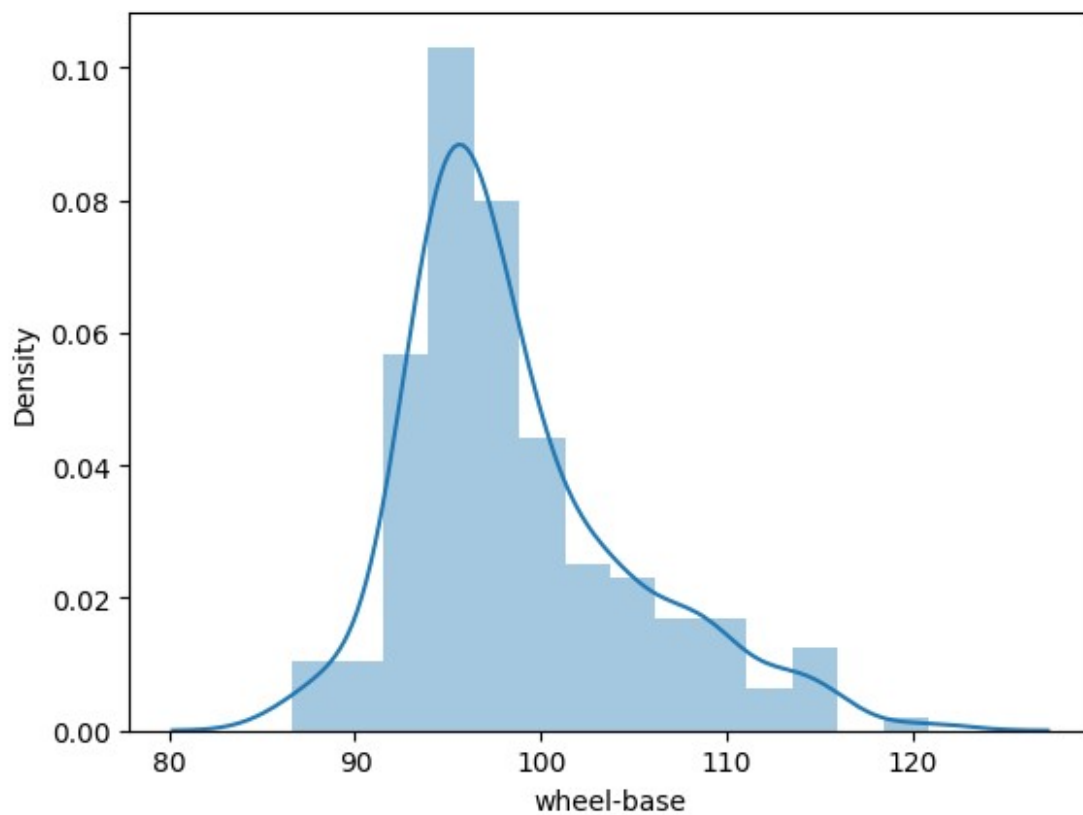
```

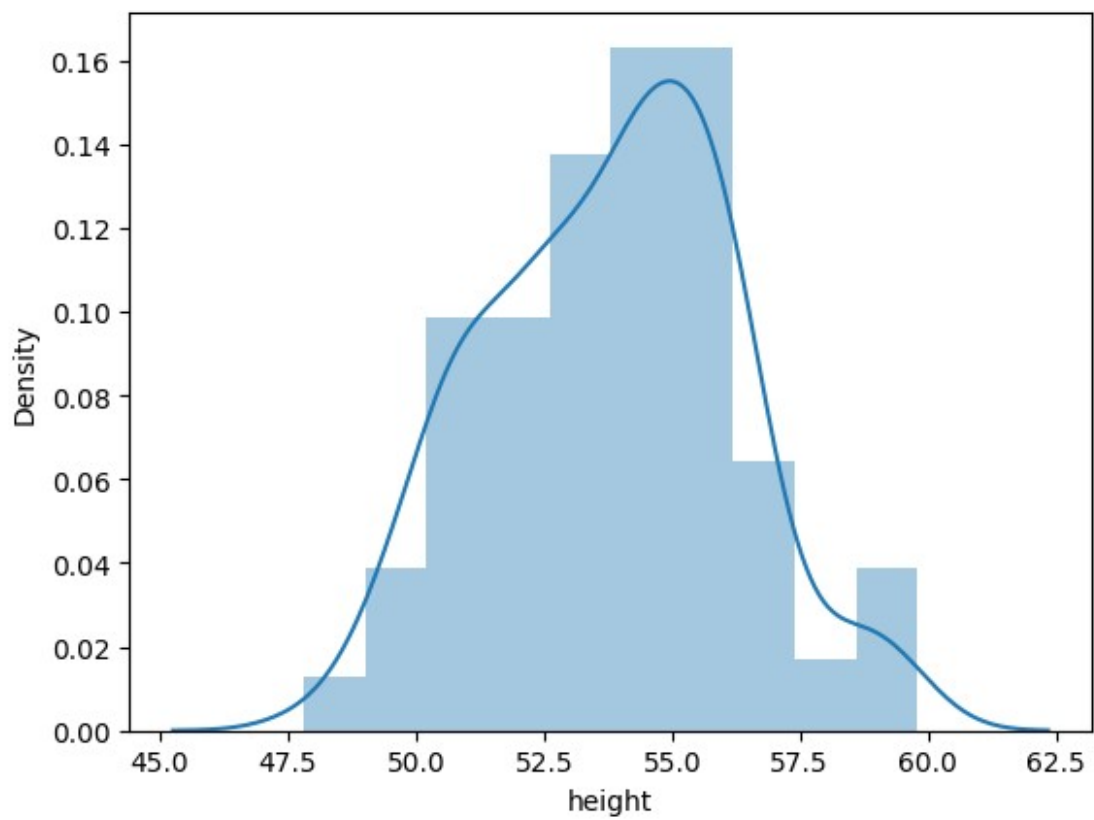
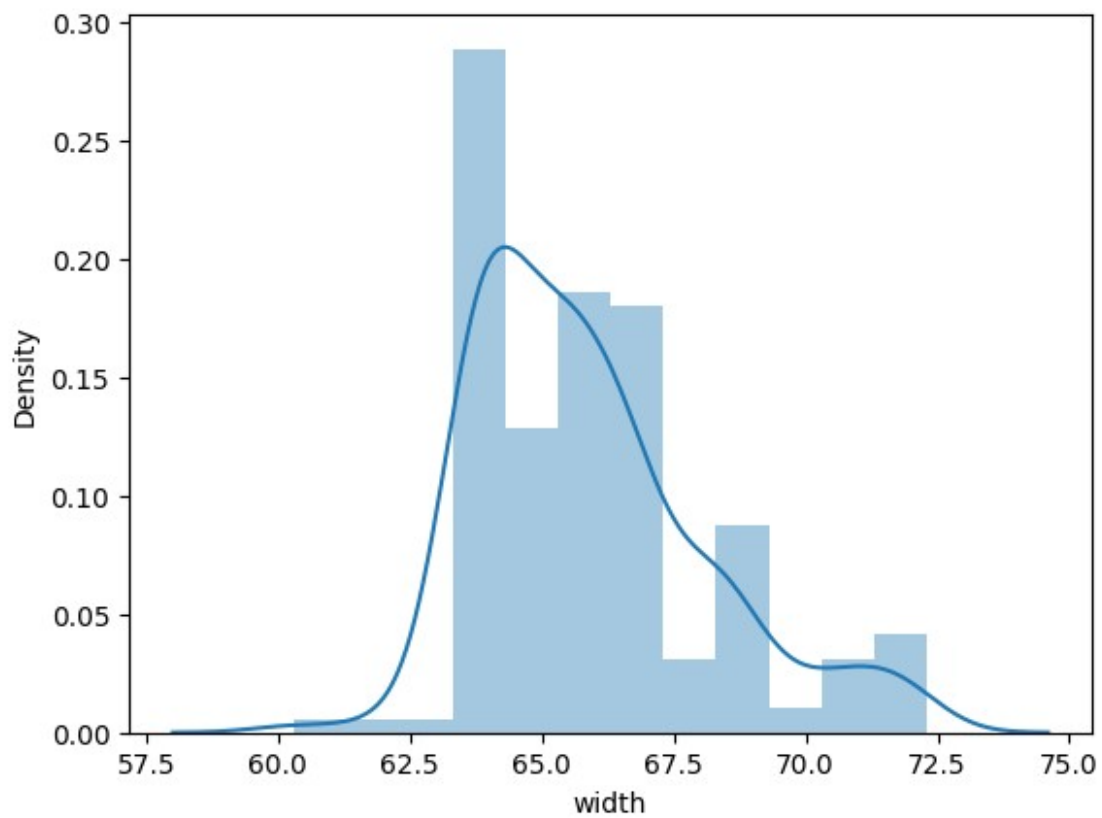
```
Index(['symboling', 'normalized-losses', 'wheel-base', 'length',  
      'width',  
      'height', 'curb-weight', 'engine-size', 'bore', 'stroke',  
      'compression-ratio', 'horsepower', 'peak-rpm', 'city-mpg',  
      'highway-mpg'],  
      dtype='object')
```

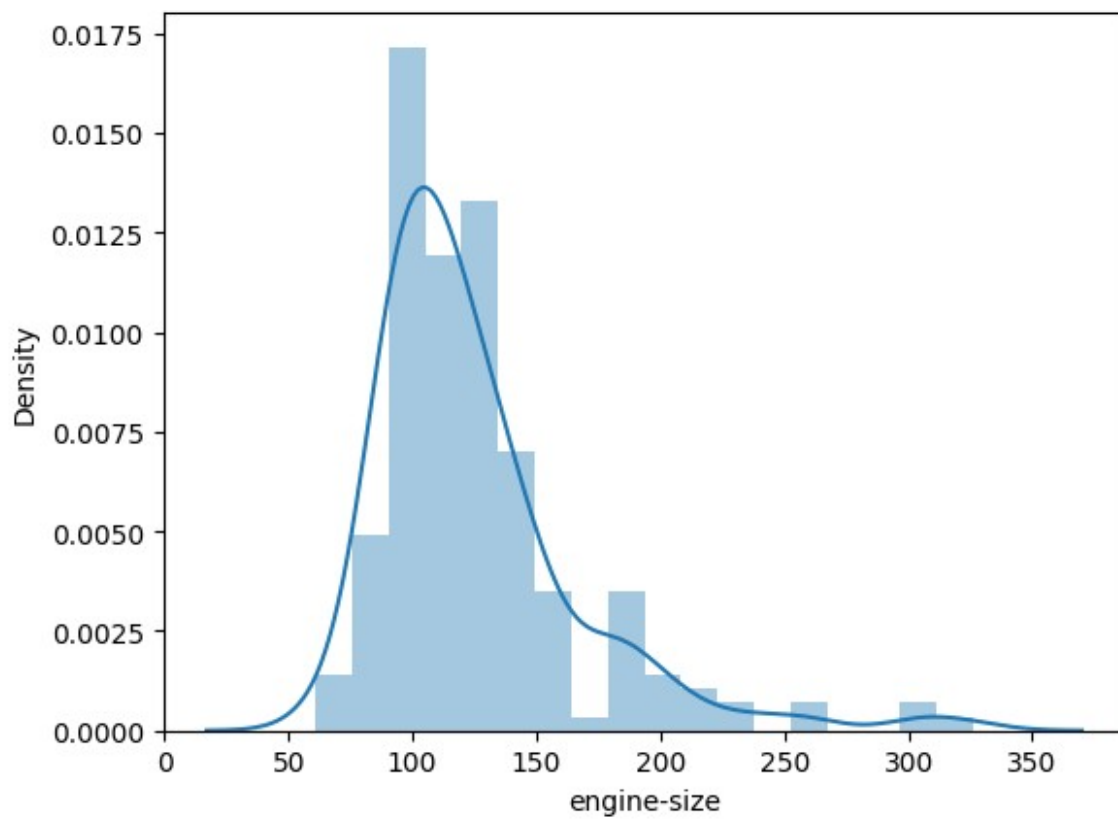
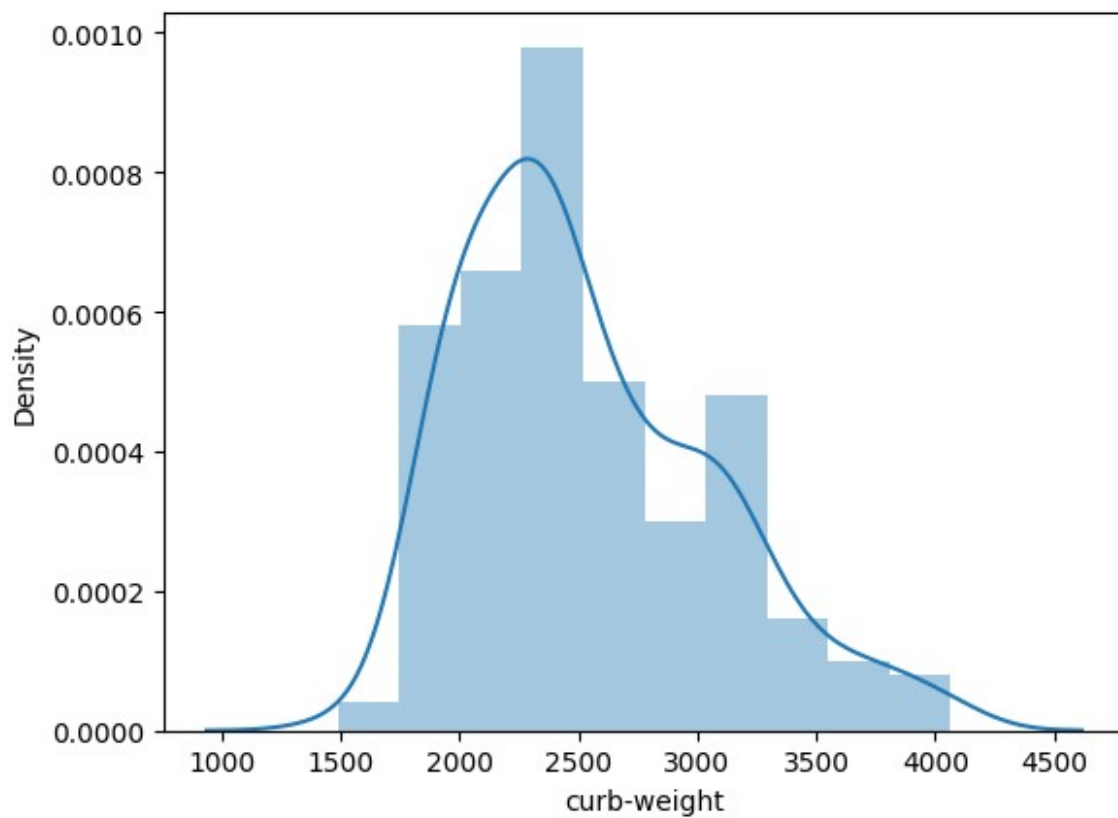
```
from scipy.stats import skew  
for cols in features[colnames]:  
    print(cols)  
    print(skew(features[cols]))  
  
plt.figure()  
sns.distplot(features[cols])  
plt.show
```

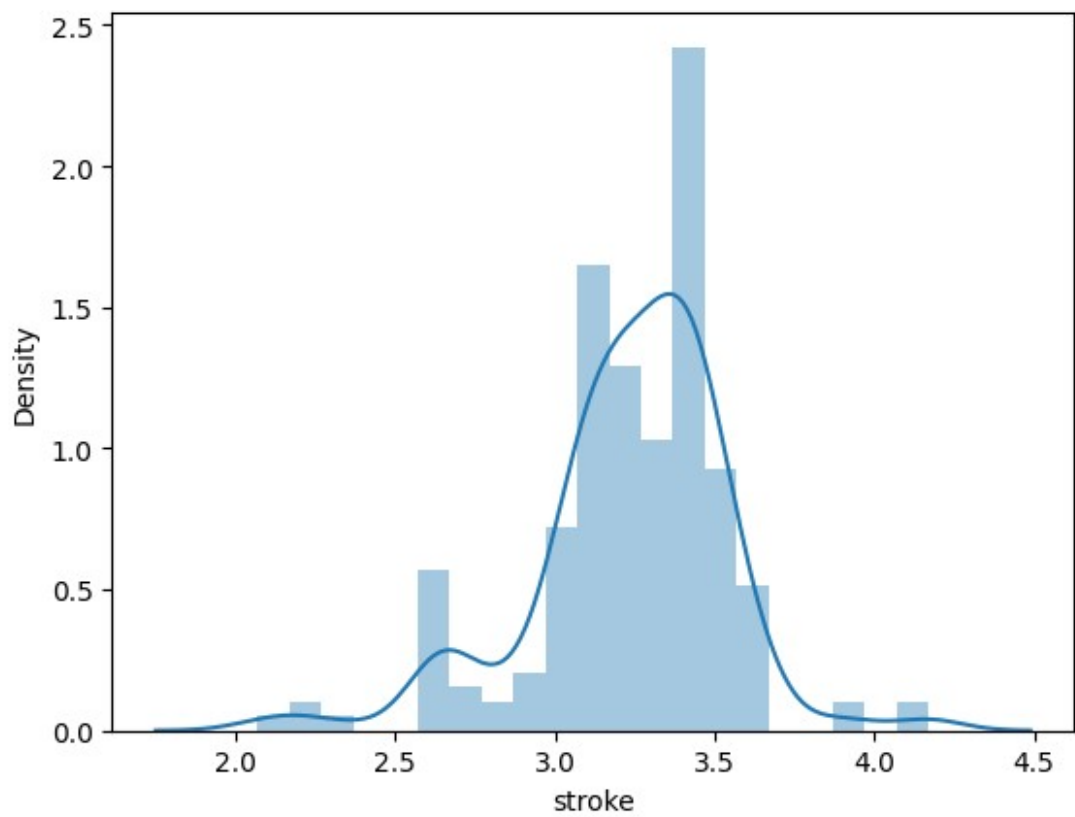
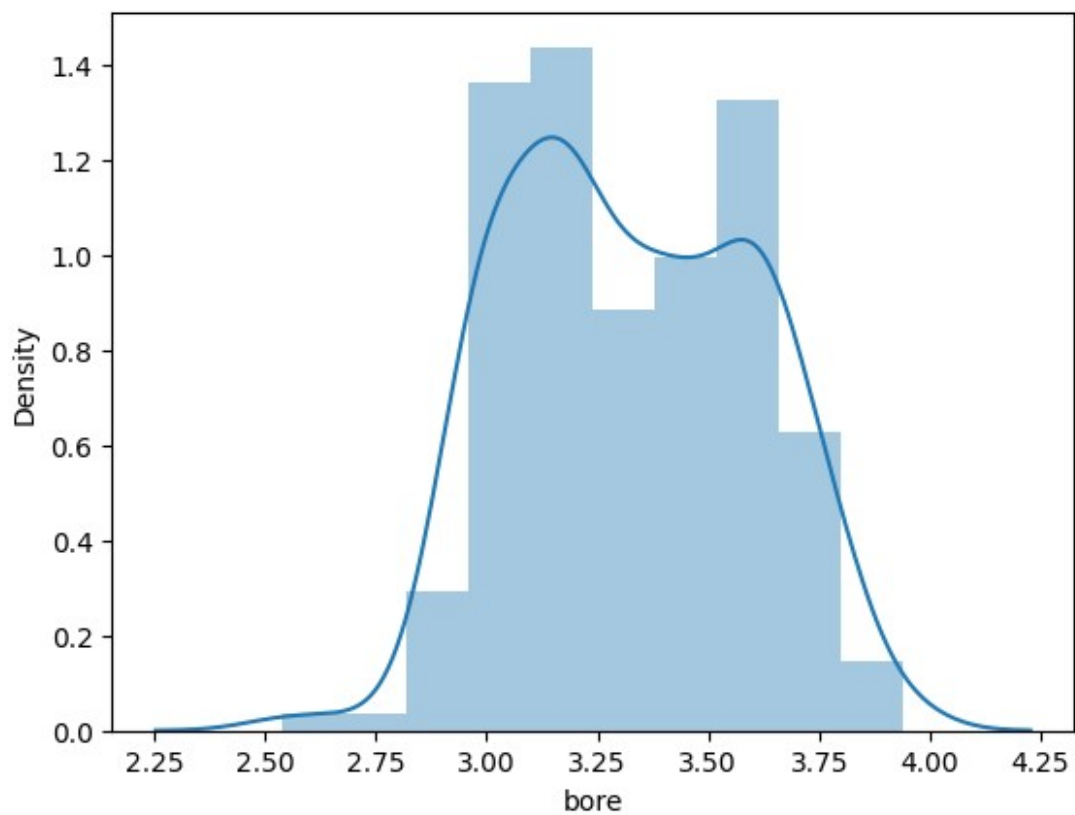
```
symboling  
0.22326592087047767  
normalized-losses  
0.848205953606264  
wheel-base  
1.0434318954680746  
length  
0.19318313729179817  
width  
0.9197264769767646  
height  
0.017443445468877522  
curb-weight  
0.7785529269563564  
engine-size  
2.0704133174547303  
bore  
0.06006761573130289  
stroke  
-0.8072553195768107  
compression-ratio  
2.520677620258808  
horsepower  
1.5819241758278482  
peak-rpm  
0.061377963186987905  
city-mpg  
0.5838226949584431  
highway-mpg  
0.45878800653395957
```

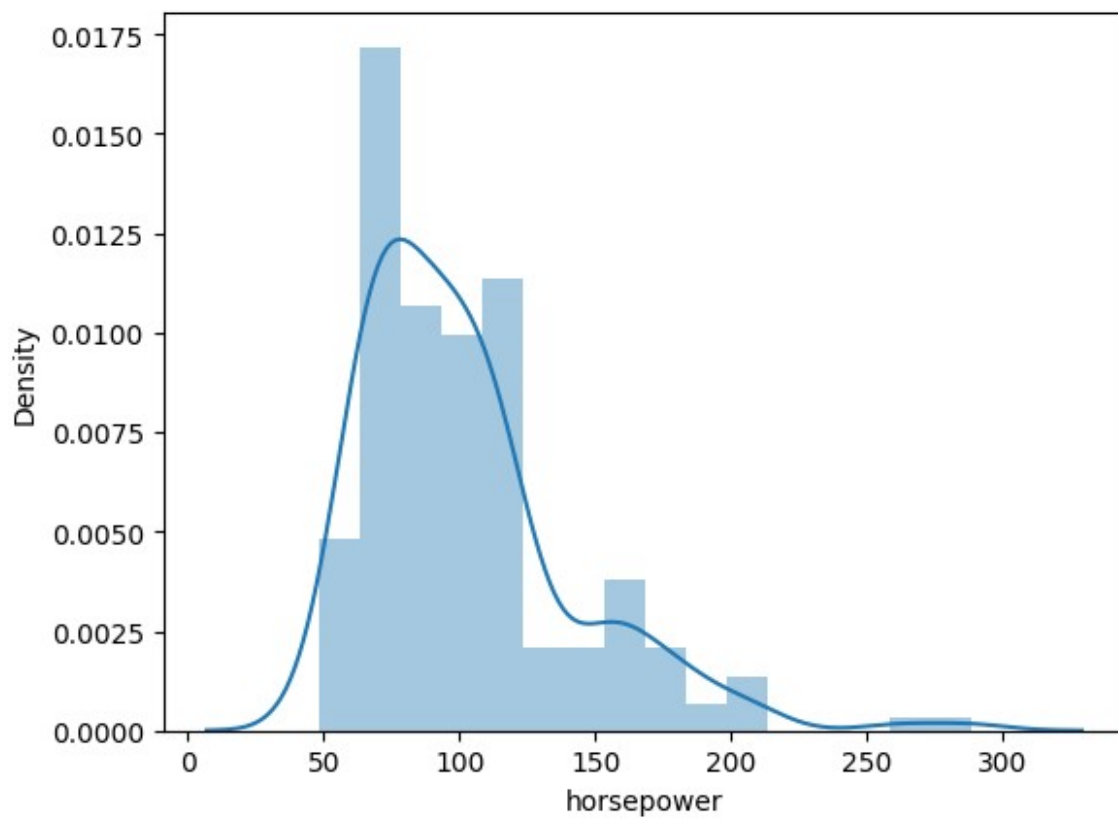
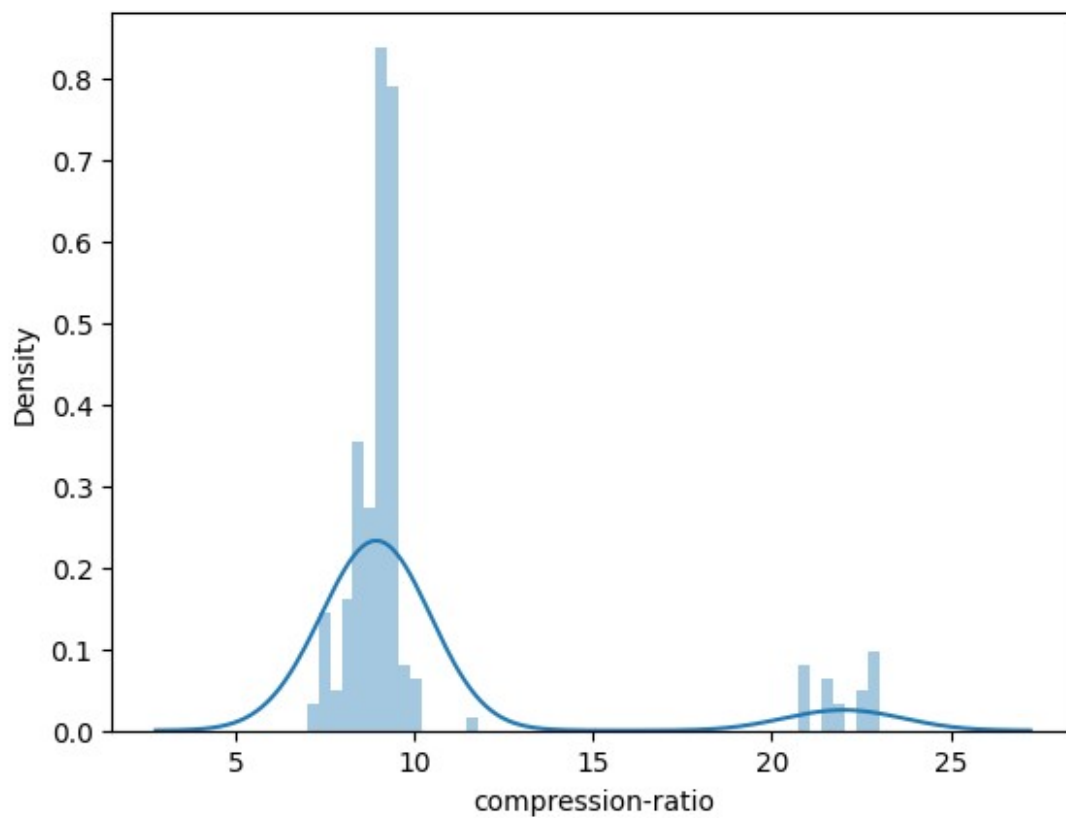


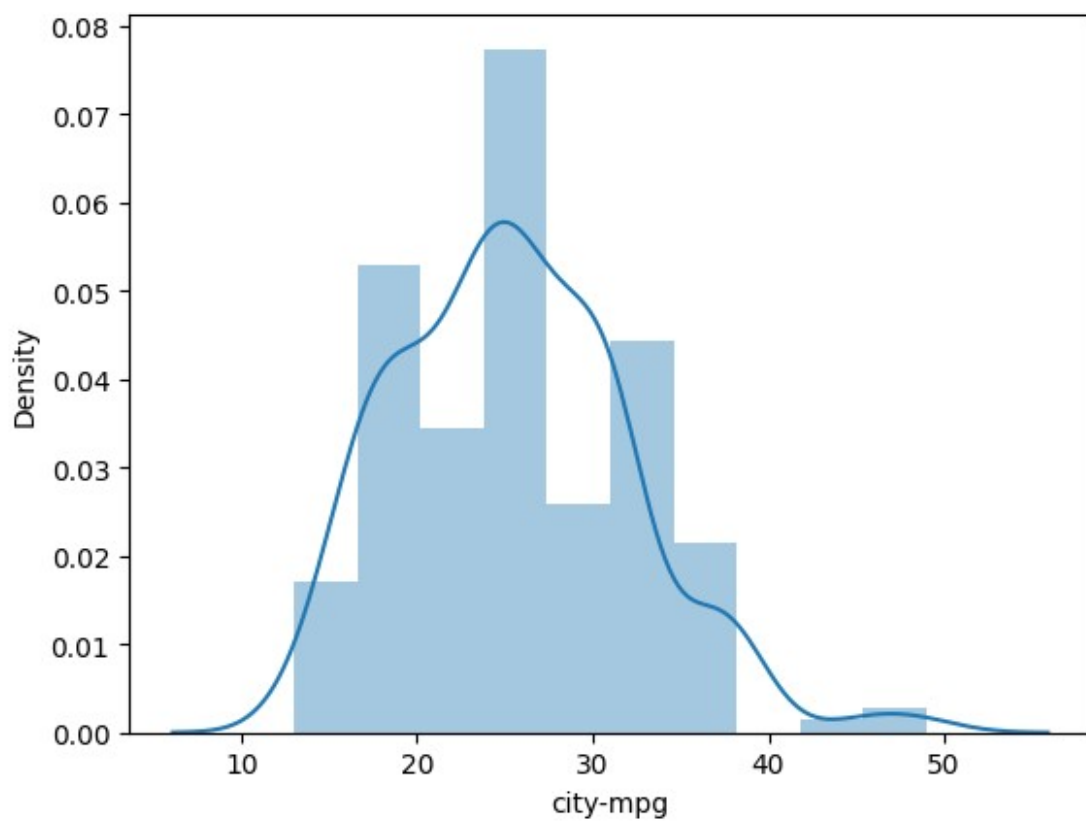
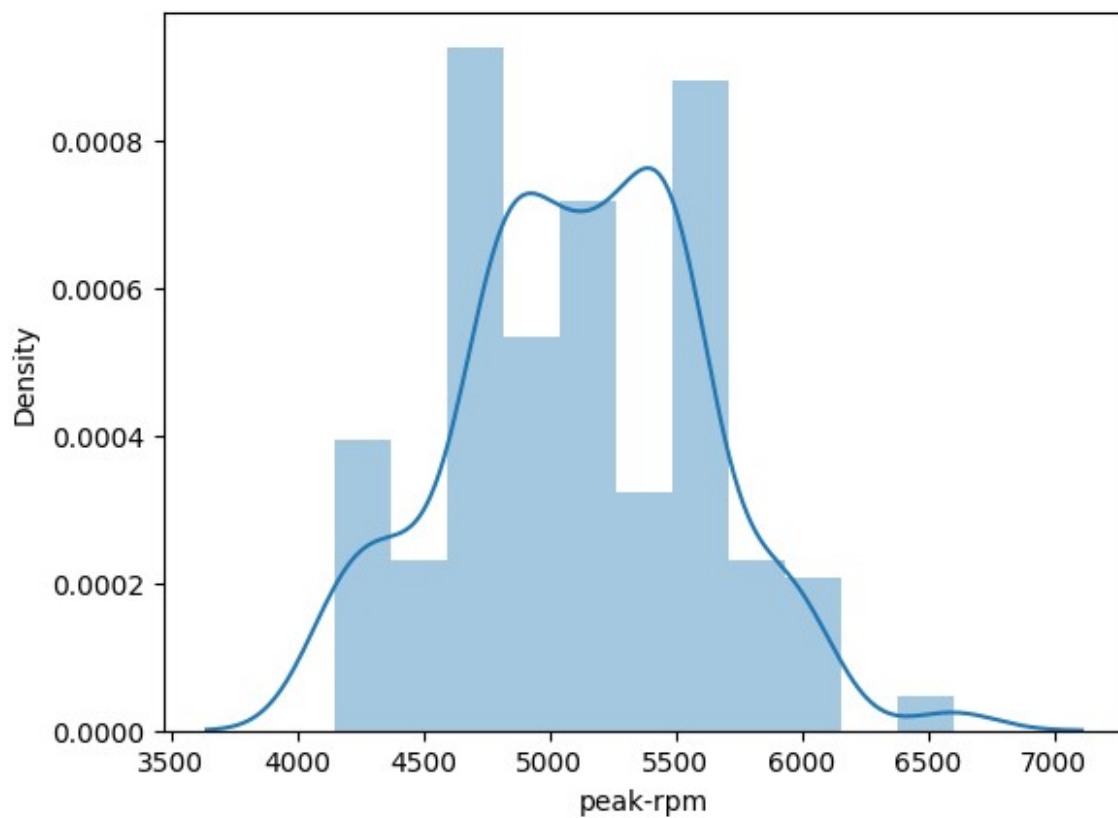


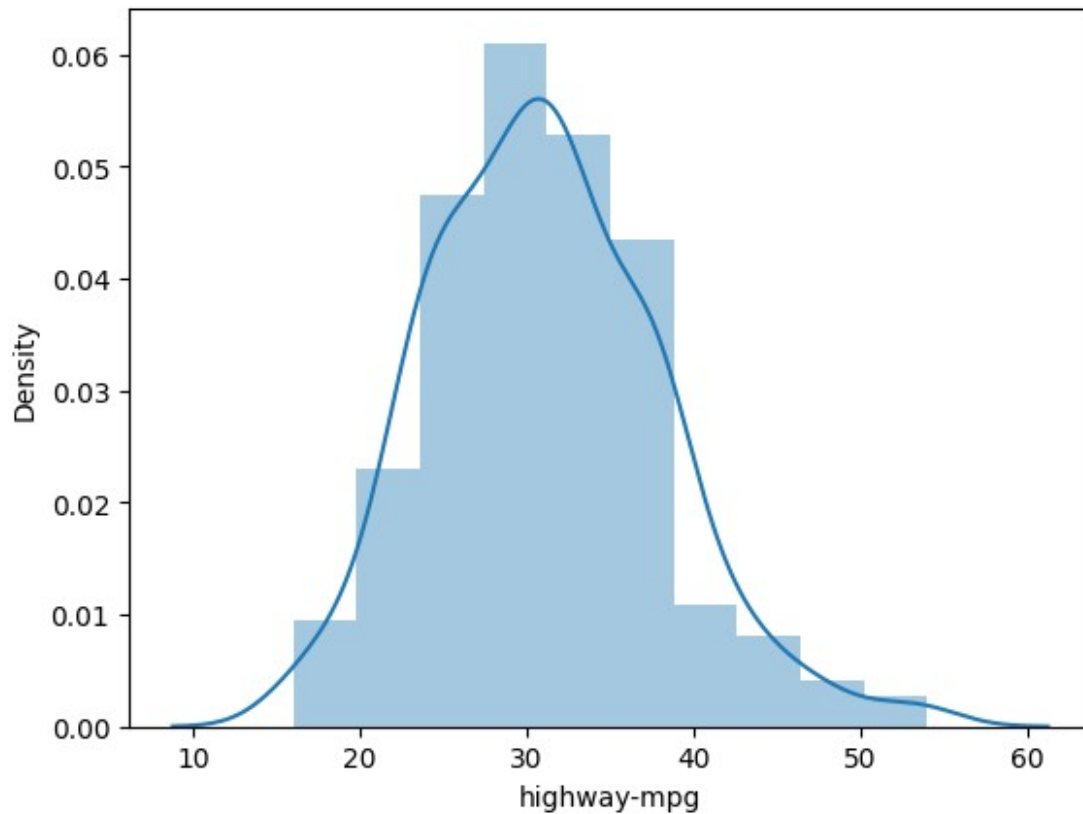












```
pd.concat([features,target],axis=1).corr()['price'].sort_values()
```

highway-mpg	-0.740975
city-mpg	-0.717768
make	-0.176597
fuel-type	-0.114502
peak-rpm	-0.100492
symboling	-0.096168
body-style	-0.066409
num-of-doors	-0.050143
num-of-cylinders	-0.002013
compression-ratio	0.075046
stroke	0.079901
engine-type	0.100935
normalized-losses	0.130112
height	0.144026
aspiration	0.191240
engine-location	0.333285
fuel-system	0.524793
bore	0.539382
wheel-base	0.578720
drive-wheels	0.582521
wheel-base	0.585277
length	0.683920

```
width          0.727942
horsepower     0.778684
curb-weight    0.811959
engine-size    0.857816
price          1.000000
Name: price, dtype: float64
```

```
from scipy import stats
```

```
features['highway-mpg'] = np.cbrt(features['highway-mpg'])
print(skew(features['highway-mpg']))
```

```
features['city-mpg'] = np.cbrt(features['city-mpg'])
print(skew(features['city-mpg']))
```

```
features[' wheel-base'] = np.cbrt(features['wheel-base'])
print(skew(features['wheel-base']))
```

```
features['width'] = np.cbrt(features['width'])
print(skew(features['width']))
```

```
features['horsepower'] = np.cbrt(features['horsepower'])
print(skew(features['horsepower']))
```

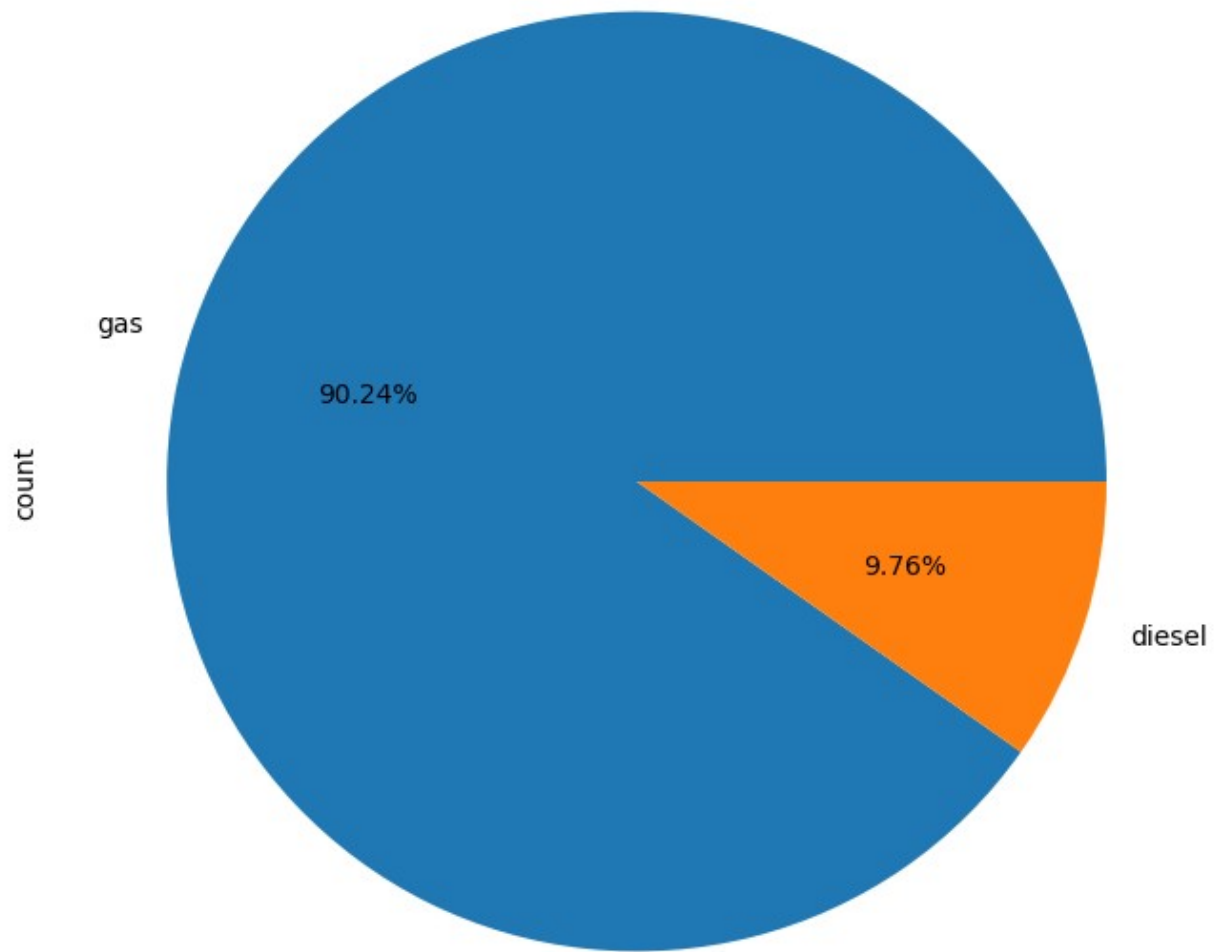
```
features['curb-weight'] = np.cbrt(features['curb-weight'])
print(skew(features['curb-weight']))
```

```
features['engine-size'] = np.cbrt(features['engine-size'])
print(skew(features['engine-size']))
```

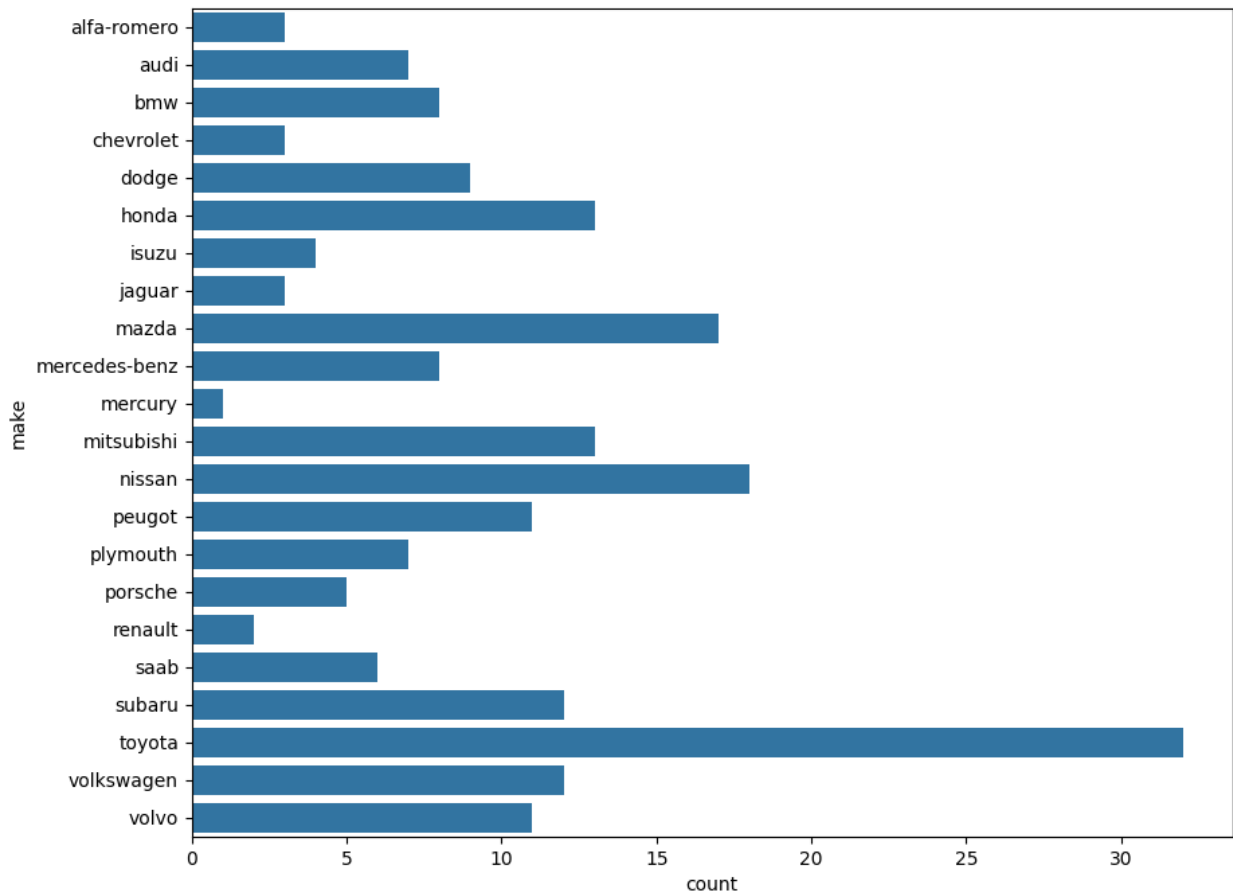
```
0.003782440230506038
0.12004656888819494
1.0434318954680746
0.8630336796803285
0.8889013019724904
0.5048593222593741
1.322843067231212
```

```
df['fuel-
type'].value_counts().plot(figsize=(8,8),kind='pie',autopct='%.2f%%')
```

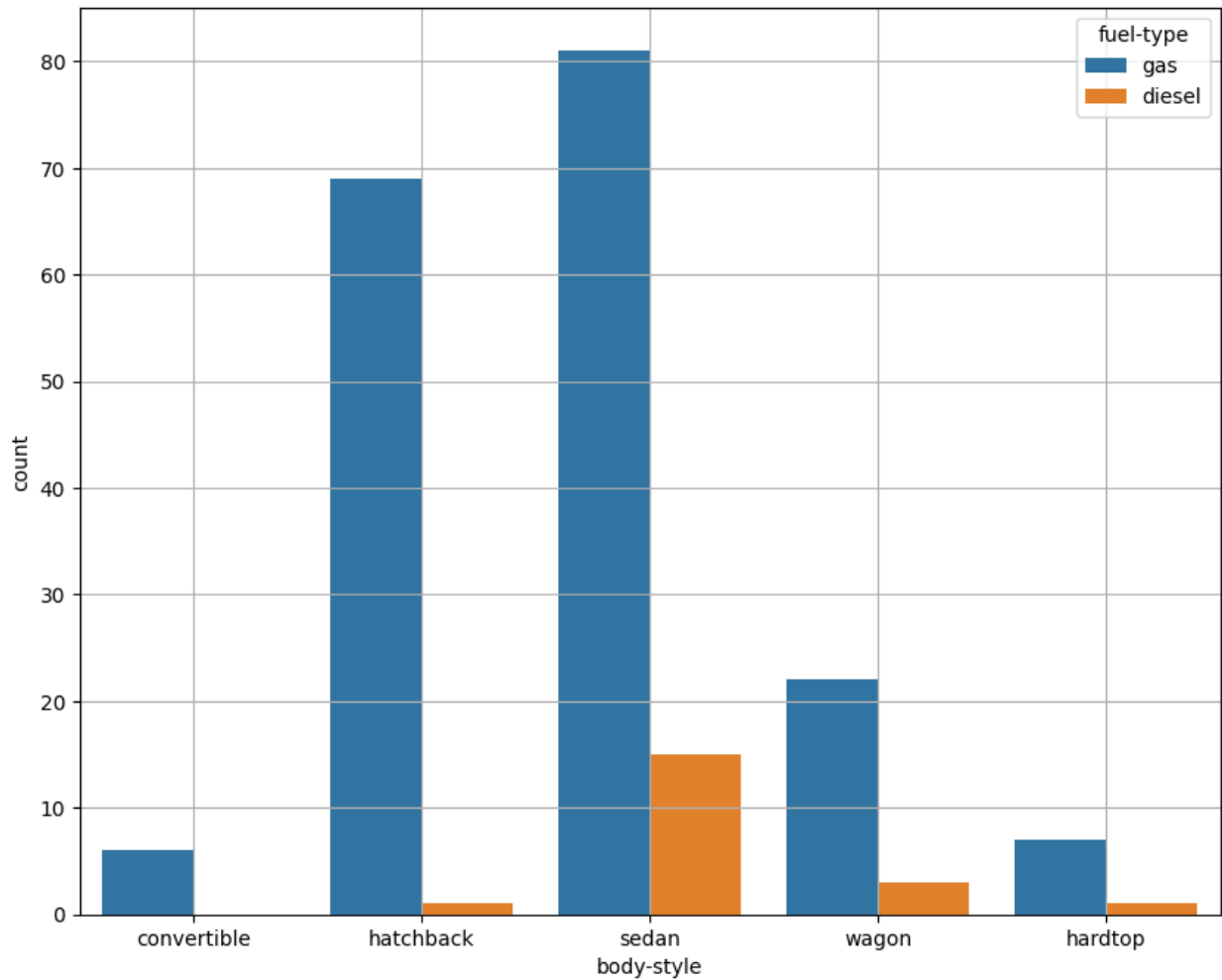
```
<Axes: ylabel='count'>
```



```
plt.figure(figsize=(10,8))  
sns.countplot(data=df,y='make')  
plt.show()
```

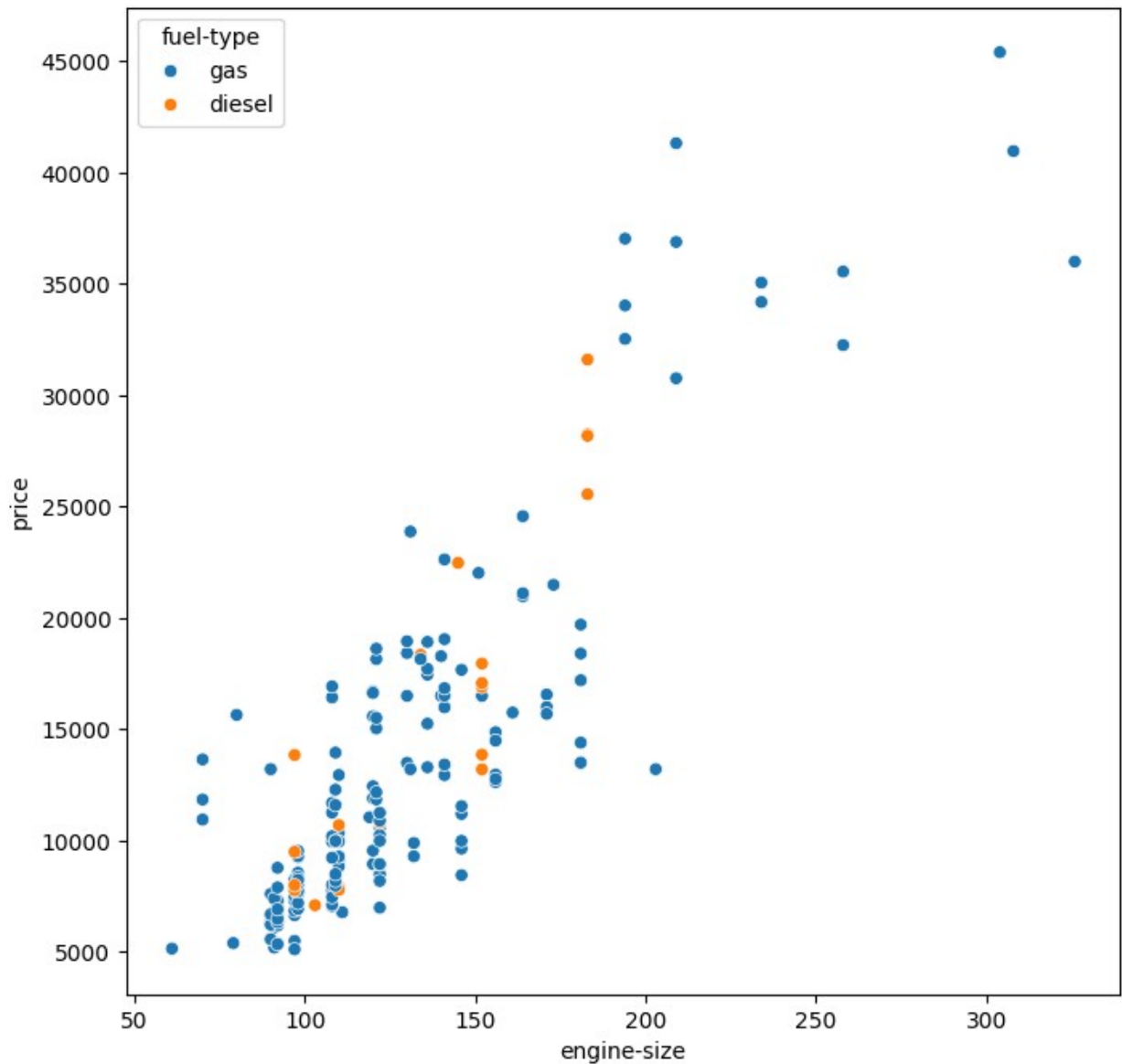



```
plt.figure(figsize=(10,8))  
sns.countplot(data=df,x='body-style',hue='fuel-type')  
plt.grid(True)
```



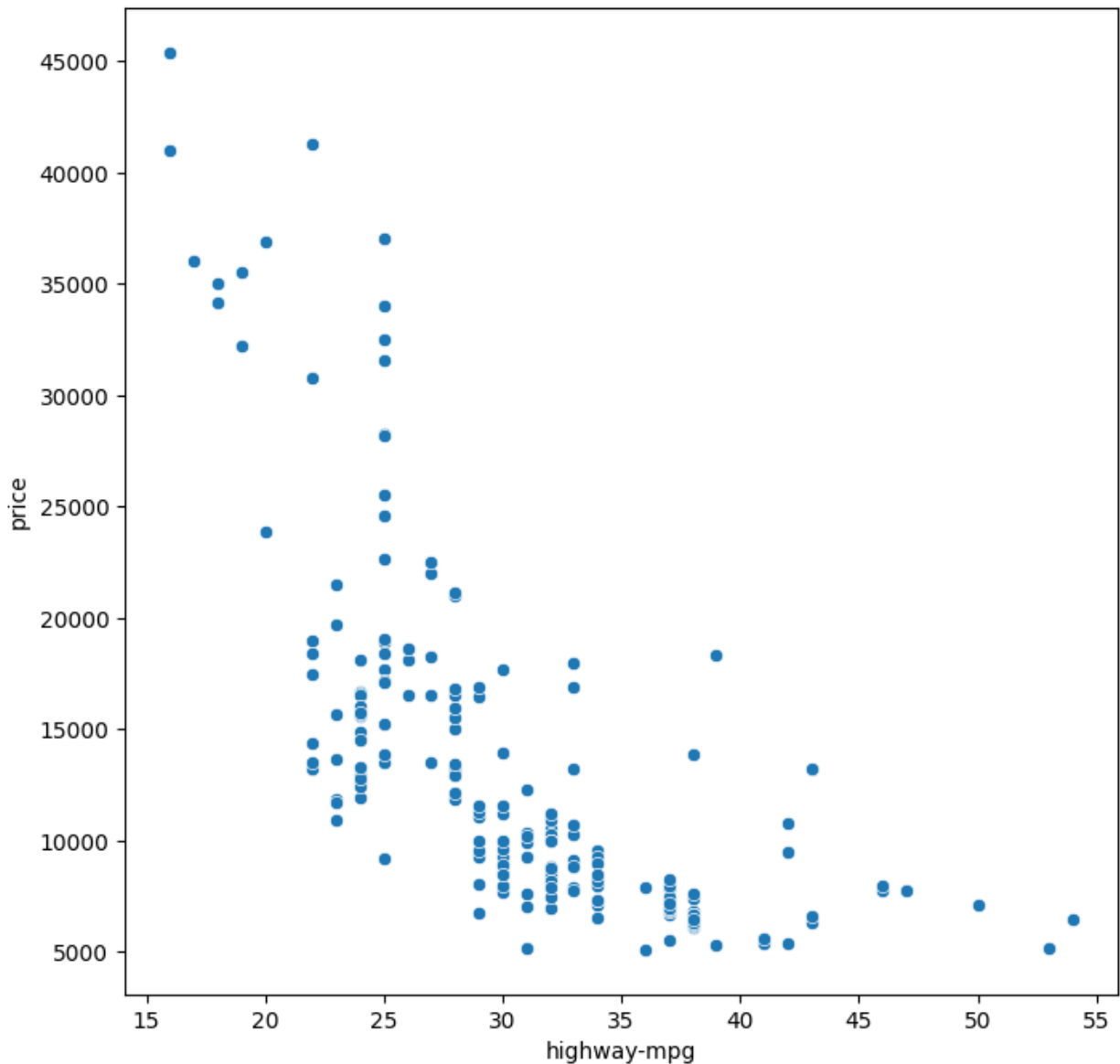
```
plt.figure(figsize=(8,8))
sns.scatterplot(y='price',x='engine-size',data=df,hue='fuel-type')
plt.show
```

```
<function matplotlib.pyplot.show(close=None, block=None)>
```



```
plt.figure(figsize=(8,8))
sns.scatterplot(data=df,x='highway-mpg',y='price')
plt.show

<function matplotlib.pyplot.show(close=None, block=None)>
```



```
plt.figure(1)
plt.subplot(2,2,1)
df['engine-
type'].value_counts(normalize=True).plot(figsize=(10,8),kind='bar',col
or='red')
plt.title('Number of engine type frequency diagram')
plt.xlabel('Number of engine type')
plt.ylabel('engine type')

plt.subplot(2,2,2)
df['drive-
wheels'].value_counts(normalize=True).plot(figsize=(10,8),kind='bar',c
olor='green')
plt.title('Number of drive wheels frequency diagram')
```

```

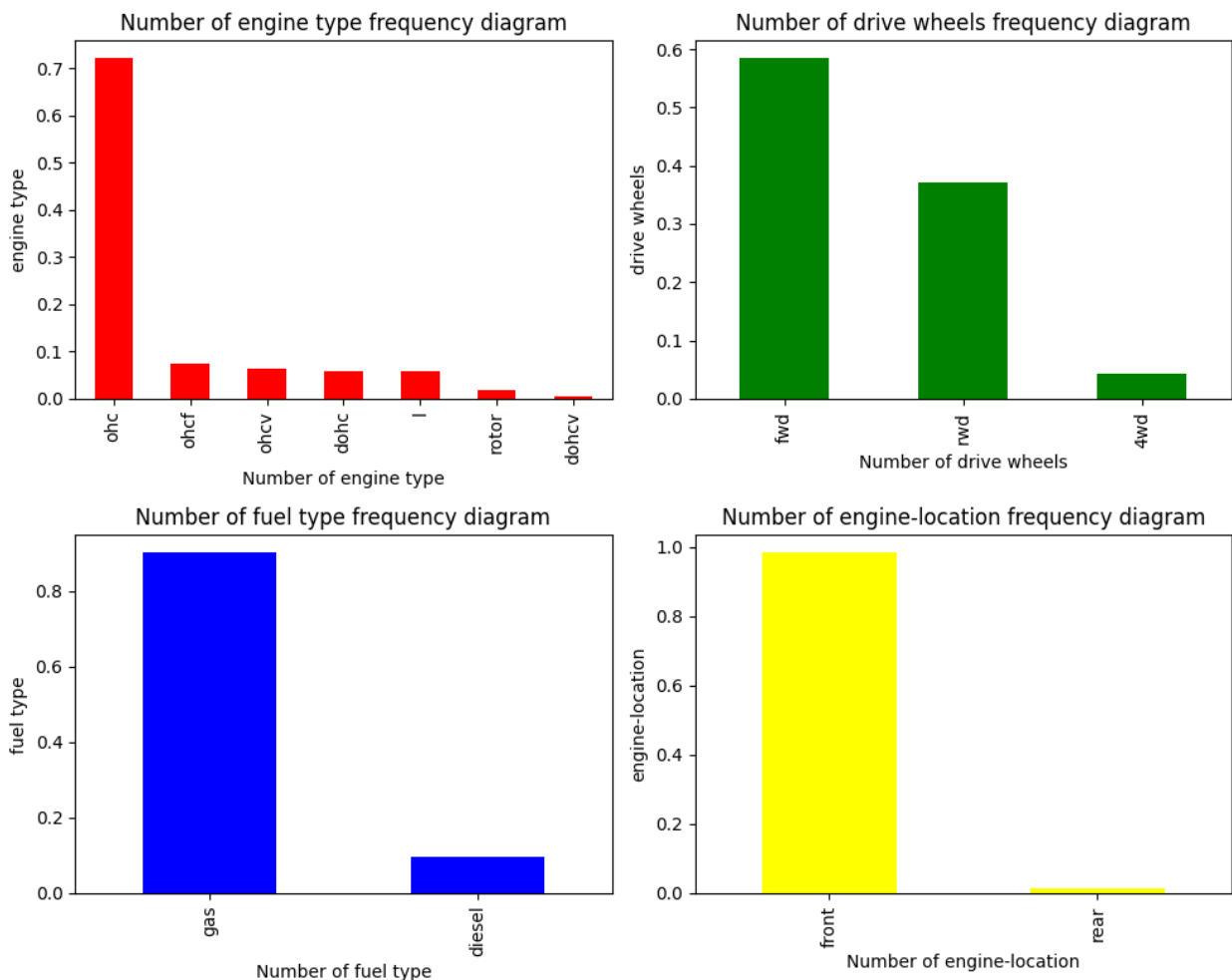
plt.xlabel('Number of drive wheels')
plt.ylabel('drive wheels')

plt.subplot(2,2,3)
df['fuel-
type'].value_counts(normalize=True).plot(figsize=(10,8),kind='bar',col
or='blue')
plt.title('Number of fuel type frequency diagram')
plt.xlabel('Number of fuel type')
plt.ylabel('fuel type')

plt.subplot(2,2,4)
df['engine-
location'].value_counts(normalize=True).plot(figsize=(10,8),kind='bar'
,color='yellow')
plt.title('Number of engine-location frequency diagram')
plt.xlabel('Number of engine-location')
plt.ylabel('engine-location')

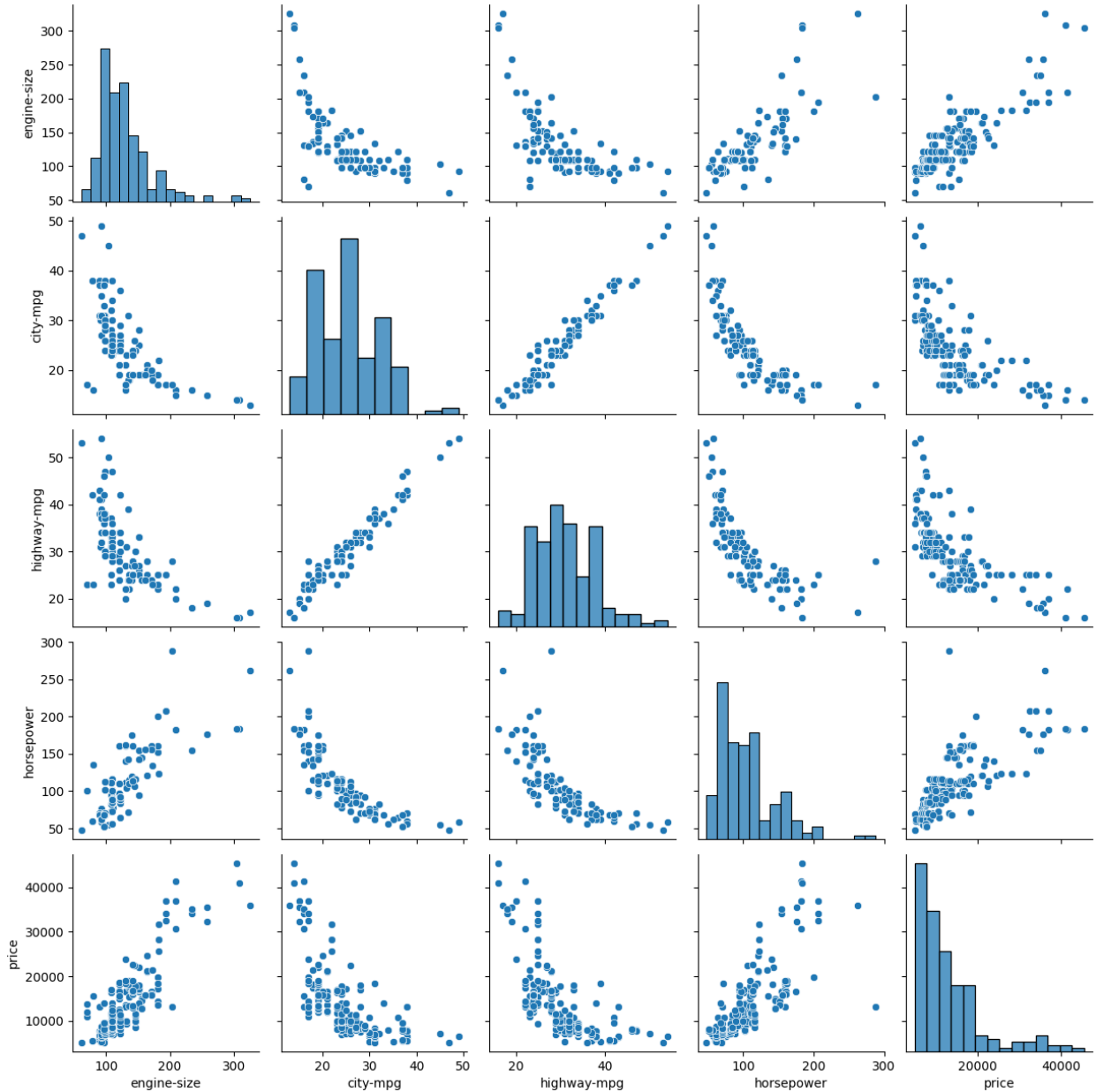
plt.tight_layout()

```

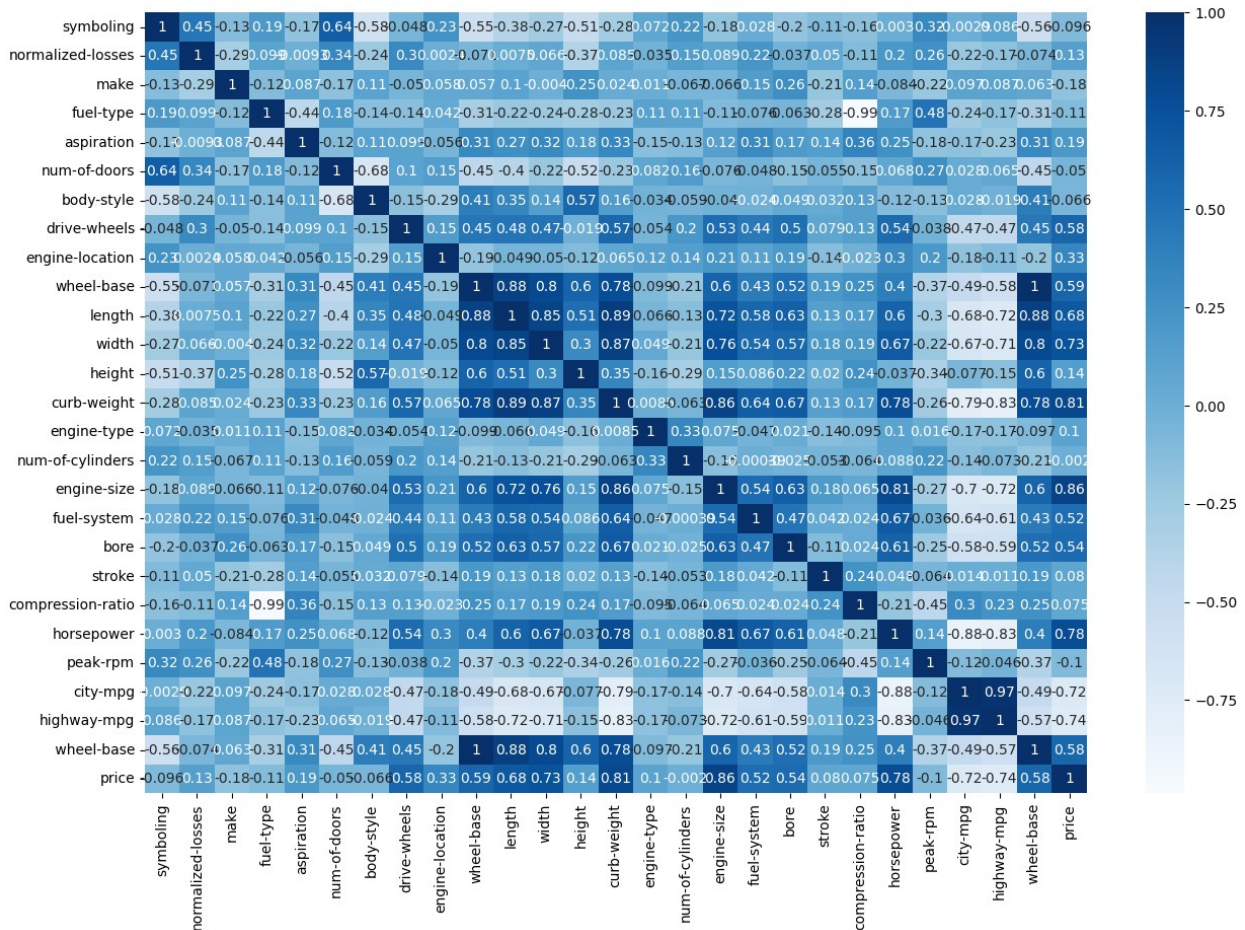


```
sns.pairplot(df[['engine-size', 'body-style', 'city-mpg', 'highway-mpg', 'horsepower', 'price']])
```

<seaborn.axisgrid.PairGrid at 0x7bb3659f9640>



```
plt.figure(figsize=(15,10))
sns.heatmap(pd.concat([features, target], axis=1).corr(), annot=True,
            cmap='Blues')
plt.show()
```



```
cols = features.select_dtypes(['object']).columns
cols
```

```
Index([], dtype='object')
```

```
from sklearn.preprocessing import OrdinalEncoder
oe = OrdinalEncoder()
features[cols] = oe.fit_transform(features[cols])
features.head()
```

```
{"type": "dataframe", "variable_name": "features"}
```

```
from sklearn.model_selection import train_test_split
xtrain, xtest, ytrain, ytest =
train_test_split(features, target, random_state=2, test_size=0.2)
```

```
from sklearn.linear_model import LinearRegression
lr = LinearRegression()
lr.fit(xtrain, ytrain)
ypred = lr.predict(xtest)
```

```

from sklearn.metrics import mean_squared_error, r2_score
mse = mean_squared_error(ytest,ypred)
r2 = r2_score(ytest,ypred)
print('mean_squared_error : ',mse)
print('r2_score : ',r2)

mean_squared_error :  18179265.100488544
r2_score :  0.6678136852335932

train = lr.score(xtrain,ytrain)
test = lr.score(xtest,ytest)
print(f'Training_Accuracy : {train}\nTesting_Accuracy : {test}')

Training_Accuracy : 0.8776297293657729
Testing_Accuracy : 0.6678136852335932

plt.scatter(ytest,ypred)
plt.show()

```

